

Solution Code - Module 3-Core Java - InsertUpdateJDBC

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q32_InsertUpdateJDBC.java

```
1 | import java.sql.Connection;
2 | import java.sql.DriverManager;
3 | import java.sql.PreparedStatement;
4 | import java.sql.ResultSet;
5 | import java.sql.Statement;
6 |
7 | // Student DAO (Data Access Object) Class
8 | class StudentDAO {
9 |     private final String dbUrl;
10 |
11 |     public StudentDAO(String dbUrl) {
12 |         this.dbUrl = dbUrl;
13 |     }
14 |
15 |     private Connection getConnection() throws Exception {
16 |         return DriverManager.getConnection(dbUrl);
17 |     }
18 |
19 |     // Initialize database structure
20 |     public void initDatabase() throws Exception {
21 |         try (Connection conn = getConnection();
22 |             Statement stmt = conn.createStatement()) {
23 |             stmt.execute("DROP TABLE IF EXISTS students;");
24 |             stmt.execute("CREATE TABLE IF NOT EXISTS students ("
25 |                 + "id INTEGER PRIMARY KEY AUTOINCREMENT, "
26 |                 + "name TEXT NOT NULL, "
27 |                 + "grade TEXT"
28 |                 + ");");
29 |         }
30 |     }
31 |
32 |     // Method to insert a new student using PreparedStatement
33 |     public int insertStudent(String name, String grade) {
34 |         String insertSql = "INSERT INTO students (name, grade) VALUES (?, ?);";
35 |         try (Connection conn = getConnection();
36 |             PreparedStatement pstmt = conn.prepareStatement(insertSql, Statement.RETURN_GENERATED_KEYS))
37 |         {
38 |             pstmt.setString(1, name);
39 |             pstmt.setString(2, grade);
40 |             pstmt.executeUpdate();
41 |
42 |             try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
```

Solution Code - Module 3-Core Java - InsertUpdateJDBC

```
43 |         if (generatedKeys.next()) {
44 |             return generatedKeys.getInt(1); // Return generated ID
45 |         }
46 |     }
47 | } catch (Exception e) {
48 |     System.err.println("Error inserting student: " + e.getMessage());
49 | }
50 | return -1;
51 | }
52 |
53 | // Method to update student details using PreparedStatement
54 | public boolean updateStudent(int id, String newName, String newGrade) {
55 |     String updateSql = "UPDATE students SET name = ?, grade = ? WHERE id = ?";
56 |     try (Connection conn = getConnection();
57 |         PreparedStatement pstmt = conn.prepareStatement(updateSql)) {
58 |
59 |         pstmt.setString(1, newName);
60 |         pstmt.setString(2, newGrade);
61 |         pstmt.setInt(3, id);
62 |
63 |         int rowsAffected = pstmt.executeUpdate();
64 |         return rowsAffected > 0;
65 |     } catch (Exception e) {
66 |         System.err.println("Error updating student: " + e.getMessage());
67 |     }
68 |     return false;
69 | }
70 |
71 | // Method to print all students
72 | public void printAllStudents() {
73 |     try (Connection conn = getConnection();
74 |         Statement stmt = conn.createStatement();
75 |         ResultSet rs = stmt.executeQuery("SELECT * FROM students;")) {
76 |
77 |         System.out.println("Current Students Database Entries:");
78 |         while (rs.next()) {
79 |             System.out.println("    ID: " + rs.getInt("id") + ", Name: " + rs.getString("name") + ",
Grade: " + rs.getString("grade"));
80 |         }
81 |     } catch (Exception e) {
82 |         System.err.println("Error reading students: " + e.getMessage());
83 |     }
84 | }
85 | }
86 |
87 | public class Q32_InsertUpdateJDBC {
88 |     public static void main(String[] args) {
89 |         String dbUrl = "jdbc:sqlite:students.db";
90 |         System.out.println("=== JDBC Insert and Update Operations Demo ===");
91 |     }
```

Solution Code - Module 3-Core Java - InsertUpdateJDBC

```
92 |         try {
93 |             Class.forName("org.sqlite.JDBC");
94 |             StudentDAO dao = new StudentDAO(dbUrl);
95 |             dao.initDatabase();
96 |
97 |             // 1. Insert new student records
98 |             System.out.println("Inserting new students...");
99 |             int id1 = dao.insertStudent("David", "B");
100 |             int id2 = dao.insertStudent("Emma", "A");
101 |             dao.printAllStudents();
102 |
103 |             // 2. Update student details
104 |             System.out.println("\nUpdating student with ID " + id1 + " to 'David Smith' and grade
'A'...");
105 |             boolean success = dao.updateStudent(id1, "David Smith", "A");
106 |             if (success) {
107 |                 System.out.println("Update successful!");
108 |             } else {
109 |                 System.out.println("Update failed.");
110 |             }
111 |
112 |             dao.printAllStudents();
113 |         } catch (Exception e) {
114 |             e.printStackTrace();
115 |         }
116 |     }
117 | }
```